



UNIVERSITY OF GENEVA
Department of Informatics

Hoare Logic and Function Calls

Software Modeling & Verification

Fitzwilliam Darcy

October 13, 2025

Table of Contents

1	Introduction	2
2	Quick recap: Hoare triples	2
3	Function contracts	2
4	Weakest precondition for function calls	2
5	Example: the recursive sum function	3
6	Termination — variants and invariants	4
7	The equivalence theorem (operational vs axiomatic semantics)	4
7.1	Useful lemmas	4
7.2	Proof of the theorem (by structural induction on program c)	5
7.2.1	Base case: assignment $x := t$	5
7.2.2	Inductive step: conditional if b then c_1 else c_2 endif	5
7.2.3	Other constructs	6
8	Putting it together: verification workflow	6

1 Introduction

These notes expand and clarify the lecture on Hoare logic for function calls, partial correctness, and termination. The goal is a clean, readable Quarto (*.qmd) document you can render or adapt. I used your original notes as the foundation and added explanations and a full proof of the theorem relating operational and axiomatic semantics.

2 Quick recap: Hoare triples

A Hoare triple has the form

$$\{P\}; c; \{Q\}$$

meaning: if the precondition P holds before executing code c , and c terminates, then the postcondition Q holds afterwards.

- P and Q are assertions (first-order logic formulae) over program variables.
- We distinguish *partial correctness* (the above statement assuming termination) from *total correctness* (partial correctness + termination proof).

3 Function contracts

A function f with parameters x can be associated with a contract:

$$\{\phi(x)\}; f; \{\psi(x, \text{res})\}$$

which reads: if the caller guarantees $\phi(x)$ holds on the actual parameter values, then whenever f returns a result **res** the postcondition $\psi(x, \text{res})$ holds.

Note: ϕ depends only on the *input* parameters, ψ may depend on both inputs and the returned result.

4 Weakest precondition for function calls

Consider a simple call used as an assignment:

```
a := f(b);
```

If we have a postcondition Q that refers to a (and other variables), the *weakest precondition* (WP) that guarantees Q after the call must ensure three things:

1. The *call site* must satisfy f 's precondition: $\phi[x \leftarrow b]$.
2. There must exist a return value v such that the postcondition of f holds at the call: $\psi[x \leftarrow b, \text{res} \leftarrow v]$.
3. Replacing a by that return value v in Q should make Q true.

A convenient formalization is to introduce a fresh variable v (not used elsewhere in the program) to stand for the return value, and define

$$WP(a := f(b), ; Q); =; Q[a \leftarrow v]; \wedge; \phi[x \leftarrow b]; \wedge; \psi[x \leftarrow b, ; \text{res} \leftarrow v].$$

If instead $f(b)$ occurs inside a larger expression E (e.g. $g(f(b), y)$), then you replace that occurrence by the fresh v and proceed similarly.

Important remark: if f 's precondition is not enforced at the call site, the semantics of the program (under the axiomatic/Hoare view) is undefined — you cannot reason about f using its contract.

5 Example: the recursive sum function

We use this function from your notes (rephrased slightly):

```
let sum(n : int) =
  requires { n >= 0 }
  ensures { res = \sum_{i=1}^n i }
  if n == 0 then res := 0
  else res := n + sum(n - 1)
endif;
res
```

We want to show partial correctness of the contract

$$\{n \geq 0\}; \text{sum}; \{\text{res} = \sum_{i=1}^n i\}.$$

Sketch of WP calculation (high-level):

- WP of the `if` is the conjunction of two implications corresponding to the two branches:
 - When $n = 0$, check that $Q[\text{res} \leftarrow 0]$ holds.
 - When $n \neq 0$, check WP of the assignment $\text{res} := n + \text{sum}(n - 1)$.
- For the recursive call we introduce a fresh v for the result of $\text{sum}(n - 1)$ and require the callee precondition and postcondition:

$$\phi[n \leftarrow n - 1] \text{ i.e. } n - 1 \geq 0, \text{ and } v = \sum_{i=1}^{n-1} i.$$
- The assignment requires $n + v = \sum_{i=1}^n i$, which is true given $v = \sum_{i=1}^{n-1} i$ and algebra of sums.

Thus the solver proves the verification conditions provided $n - 1 \geq 0$ is implied by the caller precondition $n \geq 0$, which is straightforward.

6 Termination — variants and invariants

Partial correctness alone does not ensure termination. To prove *total correctness* we need a termination argument. A common approach for recursive functions or loops is to give a *variant* (also called a well-founded measure):

- A variant is an expression over the parameters/state that takes values in a well-founded set (usually the non-negative integers $\mathbb{N}$).
- We must show that every recursive call strictly decreases the variant value and that the variant is bounded below (e.g. ≥ 0).

Example: for `sum(n)` the natural variant is n . Each recursive call uses $n - 1$, and under the precondition $n \geq 0$ we get $n - 1 < n$ and $n - 1 \geq 0$, so termination follows by induction on n .

7 The equivalence theorem (operational vs axiomatic semantics)

We now explain and prove the theorem you asked to expand:

Theorem. For any program fragment c and assertions P, Q , the Hoare triple $\{P\}; c; \{Q\}$ is valid in the axiomatic (weakest-precondition) sense iff the following holds in the operational semantics sense:

For every memory/state μ , if $\mu \models P$ then executing c from μ (when it terminates) produces a state $\mu' = \text{Mem}(c, \mu)$ that satisfies Q : $\mu' \models Q$. Equivalently,

$$P \Rightarrow WP(c, Q)$$

is valid (a true formula).

This theorem formalises the agreement between the axiomatic method (WP) and the operational meaning of the code.

7.1 Useful lemmas

We will use two standard facts about substitution and valuation of expressions and formulae.

Lemma 1. For any expression a , variable x , expression t , and state μ ,

$$\text{val}(a[x \leftarrow t], ; \mu) = \text{val}(a, ; \mu[x \leftarrow \text{val}(t, \mu)]).$$

Intuitively: substituting t for x inside a and then evaluating in μ is the same as evaluating a in a state where x has been updated with the value of t in μ .

Proof sketch. Prove by structural induction on a (constants, variables, arithmetic operators). It is routine; we omit low-level details.

Lemma 2. For any formula Q , variable x , expression t , and state μ ,

$$\mu \models Q[x \leftarrow t] \quad \text{iff} \quad \mu[x \leftarrow \text{val}(t, \mu)] \models Q.$$

Proof sketch. Structural induction on Q using Lemma 1 on each arithmetic subexpression and the usual semantics of logical connectives and quantifiers (if present).

7.2 Proof of the theorem (by structural induction on program c)

We show that for every program c and assertions P, Q the following two statements are equivalent:

1. $\forall \mu.; \mu \models P \Rightarrow \mu \models WP(c, Q)$.
2. $\forall \mu.; \mu \models P \Rightarrow \text{Mem}(c, \mu) \models Q$ (assuming termination where needed).

We prove equivalence by structural induction on c .

7.2.1 Base case: assignment $x := t$

Recall $WP(x := t, Q) = Q[x \leftarrow t]$.

- (1 $\Rightarrow$ 2) Assume $\mu \models P$ and $\mu \models WP(x := t, Q)$, i.e. $\mu \models Q[x \leftarrow t]$. By Lemma 2, $\mu[x \leftarrow \text{val}(t, \mu)] \models Q$. But the operational semantics of assignment sets the new state $\mu' = \mu[x \leftarrow \text{val}(t, \mu)]$, hence $\text{Mem}(x := t, \mu) \models Q$.
- (2 $\Rightarrow$ 1) Conversely, if for all μ with $\mu \models P$ we have $\text{Mem}(x := t, \mu) \models Q$, then using that $\text{Mem}(x := t, \mu) = \mu[x \leftarrow \text{val}(t, \mu)]$, we get $\mu[x \leftarrow \text{val}(t, \mu)] \models Q$. By Lemma 2 (rewritten), $\mu \models Q[x \leftarrow t]$, i.e. $\mu \models WP(x := t, Q)$.

Hence the assignment case is proven.

7.2.2 Inductive step: conditional if b then c_1 else c_2 endif

By definition

$$WP(\text{if}; b; \text{then}; c_1; \text{else}; c_2; ; Q) = (b \Rightarrow WP(c_1, Q)) \wedge (\neg b \Rightarrow WP(c_2, Q)).$$

Assume the equivalence holds for c_1 and c_2 . We show it for the conditional:

- Suppose $\mu \models (b \Rightarrow WP(c_1, Q)) \wedge (\neg b \Rightarrow WP(c_2, Q))$ and $\mu \models P$. Evaluate b in μ . If b is true then by the conjunct we have $\mu \models WP(c_1, Q)$ and by induction hypothesis executing c_1 from μ yields a state satisfying Q . If b is false the symmetric argument for c_2 applies. Thus execution of the whole conditional produces a μ' with $\mu' \models Q$. The other direction follows similarly by using the inductive hypothesis for c_1 and c_2 .

7.2.3 Other constructs

- Sequential composition $c_1; c_2$: by definition $WP(c_1; c_2, Q) = WP(c_1, WP(c_2, Q))$ and use induction on c_1 and c_2 .
- Loops: the standard WP approach uses loop invariants and (for total correctness) a variant. The equivalence is proven assuming a correct invariant and (for total correctness) a decreasing variant; this step follows the same structural induction pattern but requires the additional invariant/variant side conditions.
- Function calls: handled by replacing the call with the function contract (pre/post) and a fresh return variable, as described earlier. The induction goes through if the callee contract is correct (or if the callee body is available, by inlining the body and reasoning recursively).

This completes the proof sketch for the main language constructs; the full formalisation follows routine structural induction steps.

8 Putting it together: verification workflow

1. Write the function contract $\{\phi\}; f; \{\psi\}$.
2. For each call site, compute the WP by introducing fresh return variables and generating verification conditions:
 - Call preconditions must hold at the caller.
 - Using the callee's postcondition and substitution, the call's contribution to the caller WP is formed.
3. For recursive calls, provide a termination variant and verification obligations that the variant decreases and stays non-negative.
4. Discharge verification conditions with a SMT solver (e.g. Alt-Ergo, Z3) or by manual proof.